

Training a Convolutional Neural Network From Scratch Using CUDA: Implementation, Profiling, and Bottleneck Analysis

Mehdi Khameedeh Salar Rahnama

Supervisor: Dr. Farshad KhunJush

Department of Computer Science and Engineering
Shiraz University

Multicore & GPU Programming, Fall 2025

February 2026

Abstract

We present **CudaNN**, a high-performance training platform for convolutional neural networks implemented entirely in CUDA C++ from first principles, bypassing standard deep learning frameworks to expose the raw underlying hardware interaction. The system supports diverse architectures—LeNet, SimpleCNN, and MLP—across MNIST, CIFAR-10, and Synthetic datasets, featuring a hand-optimized stack: host-to-device streaming with double-buffered pinned memory, asynchronous checkpointing via background threads, log-synchronization reduction, and norm-logging throttling. We perform systematic profiling on an NVIDIA GeForce RTX 3050 Laptop GPU, revealing a critical *latency-bound regime* for small-batch deep learning workloads. Our analysis demonstrates that on lightweight architectures, the dominant performance limiter is not arithmetic throughput but *kernel-launch and synchronization overhead*: extremely low occupancy kernels ($\sim 1\%$ for bias gradients) and thousands of micro-scale device-to-host transfers (~ 18 bytes/call) create a control-bound environment where the GPU remains 85% idle despite active kernel execution. This report provides a detailed case study in the systems-level challenges of bare-metal GPU programming, creating a clear distinction between compute-bound optimization (tiling, tensor cores) and latency-bound optimization (kernel fusion, launch minimization).

Keywords: CUDA, GPU Profiling, Kernel Occupancy, Latency-Bound Execution, From-Scratch Training, Control Overhead

1. Introduction

Graphics Processing Units (GPUs) drive modern deep learning by providing massive parallelism through thousands of cores. While frameworks like PyTorch and TensorFlow abstract away memory management and kernel dispatch, they often obscure the *fine-grained interaction* between host orchestration and device execution. For large models (e.g., Transformers), execution is compute-bound, utilizing the GPU’s teraflops of throughput. However, for smaller, experimental, or real-time workloads, the bottleneck often shifts to the *control plane*—a regime where the overhead of launching kernels and synchronizing data outweighs the computation itself.

In this project, we accept the challenge of building a complete neural network training engine from scratch in CUDA C++. By managing every byte of memory and writing every kernel by hand, we expose the structural inefficiencies that usually remain hidden. Our platform,

CudaNN, serves as both a functional training system and a vehicle for systems-level performance analysis.

Our primary contribution is not just the implementation, but the *characterization of the overhead-bound regime*. We show that standard optimizations like asynchronous pipelining—which are critical for maximizing throughput on large workloads—can paradoxically degrade performance on lightweight models (e.g., MLP on Synthetic data) due to the added orchestration cost. Through detailed profiling, we identify specific pathological patterns: *micro-kernels* with $\sim 1\%$ occupancy that fail to saturate the GPU’s massive parallelism, and *synchronization thrashing* caused by frequent retrieval of scalar metrics (loss, accuracy) from device memory.

Key Contributions.

1. **Bare-Metal Implementation:** A complete CUDA C++ training stack with hand-written kernels for convolution, pooling, and backpropagation, managing raw pointers and explicit memory barriers.

2. **Modular Optimization Stack:** A set of independently toggleable optimizations targeting data transfer overlap, synchronization dampening, and memory reuse.
3. **Bottleneck Analysis:** A quantitative dissection of performance limiters, identifying *kernel launch overhead* and *micro-transfers* as the primary bottlenecks for small-scale deep learning, contrasting sharply with the compute-bound nature of large-scale training.

Organization. Section 2 describes the software architecture and CUDA kernel design. Section 3 details each optimization and its expected impact. Section 4 presents the experimental methodology. Section 5 analyzes training correctness, step times, memory overhead, and GPU utilization. Section ?? provides a deep-dive into each bottleneck category. Section 7 discusses the structural implications. Section 8 concludes.

2. System Architecture

2.1. High-Level Design

CudaNN is composed of CPU-side orchestration modules and GPU-side compute modules, as shown in Figure 1.

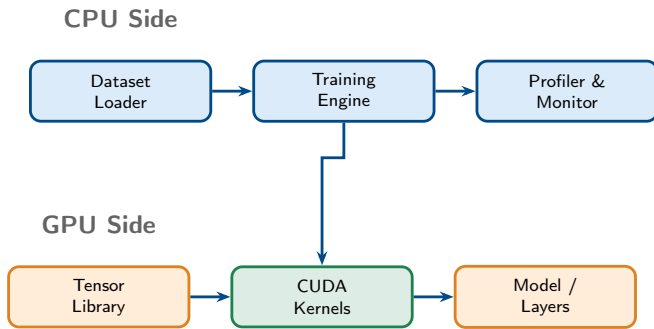


Figure 1: CudaNN system architecture.

The **CPU side** handles data loading (reading images from disk, shuffling, batching), training loop orchestration (iterating over epochs and batches, calling forward/backward/update), and profiling infrastructure (recording timestamps, GPU metrics, and exporting CSV files). The **GPU side** contains all compute-intensive components: the tensor library that manages GPU memory, the hand-written CUDA kernels, and the layer/model abstractions that wire kernels together.

2.2. Module Descriptions

- **Tensor library** (`core/tensor`) — A GPU-resident multi-dimensional array class that wraps `cudaMalloc/cudaFree` and provides shape metadata. Key feature: `Tensor::resize()` reuses existing GPU

allocations when the element count matches, avoiding expensive repeated memory allocation.

- **CUDA kernels** (`kernels/`) — 10+ hand-written kernels covering all operations. Each kernel is designed with explicit thread-block geometry: one thread per output element for convolution/linear forward, block-level parallel reductions for norms and loss, and element-wise kernels for activations.
- **Layer abstractions** (`layers/`) — C++ classes (`Conv2d`, `Linear`, `ReLU`, `MaxPool2d`, `Softmax-CrossEntropy`) with `forward()` and `backward()` interfaces. Each layer owns its parameters (weight and bias tensors) and gradient buffers.
- **Model definitions** (`model/models/`) — Three architectures: LeNet (2 conv + 3 FC), SimpleCNN (3 conv blocks + 2 FC), and MLP (3 FC layers).
- **Training engine** (`train/trainer`) — Orchestrates the training loop: for each batch, it calls forward pass, computes loss, runs backward pass, applies SGD update, and optionally collects metrics (loss, accuracy, gradient norms, weight norms).
- **Dataset loaders** (`datasets/`) — CPU-side loaders for MNIST (28×28 grayscale), CIFAR-10 (32×32×3 color), and Synthetic (randomly generated tensors for benchmarking).
- **Profiling infrastructure** (`core/profiler`, `core/system_monitor`) — Records kernel-level timing via CUDA events, GPU hardware metrics (utilization, power, memory, temperature) via NVML polling at 1 ms intervals, and per-step training metrics exported to CSV.

2.3. Model Architectures

Figure 2 shows the three architectures, which span a range of computational intensity:

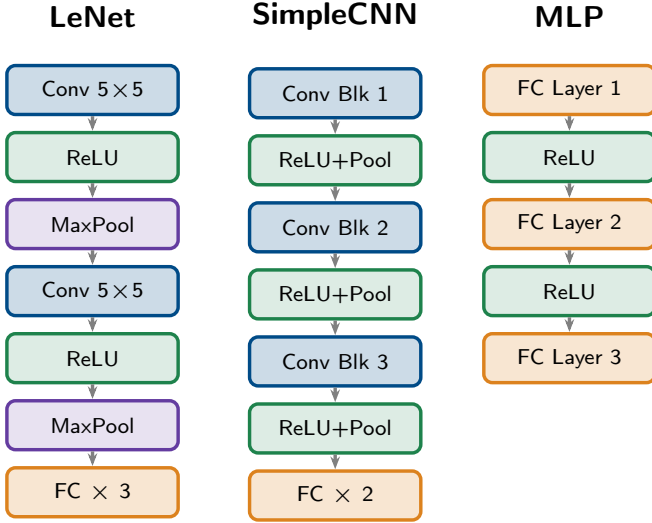


Figure 2: Three architectures spanning light (MLP) to moderate (SimpleCNN) compute.

LeNet is two convolutional layers (5×5 filters, 6 and 16 channels) with max-pooling, followed by three fully connected layers. **SimpleCNN** uses three convolutional blocks with increasing channel depth and larger spatial dimensions (designed for CIFAR-10's 32×32 input), producing the heaviest workload. **MLP** is three fully connected layers with no spatial operations, producing the lightest workload with step times of only ~ 14.8 ms.

2.4. CUDA Kernel Design & Occupancy

Table 1 details our kernel inventory. A critical insight from our implementation is the impact of *thread-block geometry* on GPU utilization. While convolution kernels (`conv2d`) are compute-dense, auxiliary kernels like bias gradients (`gradb`) and accuracy computation (`argmax`) inherently suffer from low occupancy. For example, the `conv2d_gradb` kernel reduces gradients across the batch to produce a bias vector of size equal to the usage channel count (e.g., 16 or 64). Even with a minimal block size of 256 threads, a grid size of one block utilizes only $256/24,576 \approx 1\%$ of the RTX 3050's available threads, leaving 15 of 16 SMs idle. This structural inefficiency—where the overhead of launching the kernel exceeds its execution time—is a hallmark of the latency-bound regime.

Table 1: CUDA kernel inventory with design details.

Category	Kernel	Design
Conv2D	<code>conv2d_fwd</code>	1 thread/output pixel; direct convolution
	<code>conv2d_bwd_*</code>	Separate data/weight/bias grads
Linear	<code>linear_fwd</code>	Matrix-vector product
	<code>linear_bwd_*</code>	Backend + bias reduction (low occ.)
Activation	<code>relu_fwd/bwd</code>	Element-wise; 256 threads/block
Pooling	<code>maxpool2d_*</code>	Index tracking for backward
Loss	<code>softmax_ce_*</code>	Fused softmax + cross-entropy
Reduction	<code>sumsq_partial</code>	Multi-stage parallel reduction
Optimizer	<code>sgd_update</code>	In-place parameter update
Accuracy	<code>argmax_acc</code>	Batch-wise argmax + accuracy

3. Optimization Stack

Our baseline implementation is synchronous and functionally correct. To address the latency-bound nature of the workload, we implemented six targeted optimizations. However, as our results show, the efficacy of these optimizations is heavily dependent on the "compute-to-overhead" ratio of the workload.

1. H2D Batch Streaming Pipeline. We implemented a double-buffered pipeline where Stream 1 copies batch $k + 1$ to pinned memory while Stream 0 processes batch k . *Bottleneck Insight:* This standard optimization introduces setup overhead for managing streams and events. For extremely small batches or simple models (MLP), this scheduling cost can exceed the time saved by overlapping the transfer, leading to performance regression.

2. Log-Synchronization Reduction. Baseline training extracts loss and accuracy every step via `cudaMemcpy(D2H)`, forcing a CPU-GPU sync. Our profiling revealed this to be a major bottleneck: the `simple_cnn_baseline` incurs $\sim 5,600$ D2H copies per epoch, transferring only ~ 18 bytes each. These thousands of micro-transfers act as pipeline stalls, preventing the CPU from queuing enough work to hide the latency. The optimization accumulates metrics on-device and transfers them only every N steps, dampening this "synchronization thrashing."

3. Asynchronous Checkpointing. Model serialization is offloaded to a background CPU thread, preventing disk I/O from blocking the GPU.

4. Norm-Logging Throttle. Computing L_2 norms for weights and gradients requires multiple reduction passes (using `sumsq_partial_kernel`). For LeNet, this triggered ~ 12 launches per step solely for logging, compared to just 4.8 launches for the simpler MLP. Throttling this to every 5th step eliminates $\sim 80\%$ of these overhead-heavy launches, directly addressing the "kernel-launch bound" state.

5. Tensor Storage Reuse. We minimize `cudaMalloc/cudaFree` calls by caching and resizing tensor allocations, avoiding the implicit synchronization of memory management APIs.

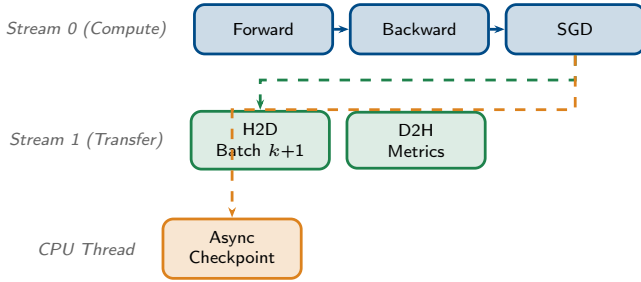


Figure 3: Optimized training pipeline designed to hide transfer latency, though limited by synchronization overhead in small workloads.

4. Experimental Setup

4.1. Hardware Platform

All experiments were conducted on a single laptop GPU. Table 2 summarizes the hardware.

Table 2: Hardware specification.

Property	Value
GPU	NVIDIA GeForce RTX 3050 Laptop
Architecture	Ampere (SM 8.6)
SMs / CUDA Cores	16 / 2048
VRAM	4 GB GDDR6 (3768 MB usable)
Clock	1740 MHz (base/boost)
OS / Toolkit	Arch Linux 6.x / CUDA 12.x

The RTX 3050 Laptop is a mid-range Ampere GPU with 16 SMs, each capable of scheduling up to 1536 threads (48 warps of 32 threads). This means the GPU can theoretically have 24,576 threads in flight simultaneously. Understanding this capacity is crucial for interpreting occupancy measurements: a kernel that launches only a single block of 256 threads achieves $256/24576 \approx 1\%$ occupancy.

4.2. Configuration Matrix

We profiled six configurations: three architectures $\times$ two modes (baseline vs. optimized), as shown in Table 3.

Table 3: Profiled configurations ($3 \times 2 = 6$ runs).

Architecture	Dataset	Batch	Epochs	LR
LeNet	MNIST ($28 \times 28 \times 1$)	64	2	0.01
SimpleCNN	CIFAR-10 ($32 \times 32 \times 3$)	64	2	0.01
MLP	Synthetic (flat vectors)	64	2	0.01

Each run used `log_every = 1` (metrics every step) and a GPU metrics polling interval of 1 ms to capture fine-grained utilization data.

4.3. Profiling Infrastructure

Each run exports five CSV files providing orthogonal views of performance:

- train_metrics.csv** — Per-step: loss, accuracy, step time (seconds), throughput (samples/sec), GPU memory usage, parameter norms, gradient norms.
- functions_summary.csv** — Aggregated memcpy statistics: call count, total time, average time, max time, total bytes, average bandwidth for each transfer type (H2D, D2H, D2D).
- gpu_metrics.csv** — Time-series GPU hardware metrics sampled at 1 ms: power draw (W), GPU utilization (%), memory utilization (%), temperature ($^{\circ}\text{C}$), SM clock (MHz).
- system_metrics.csv** — Process-level CPU usage (%) and resident set size (RSS).
- kernel_launches.csv** — Per-kernel: launch count, average duration, occupancy, grid/block dimensions.

5. Results & Analysis

5.1. Training Correctness

All architectures train successfully, showing consistent loss convergence (Figure 4). This confirms the functional correctness of our hand-written forward, backward, and optimization kernels.

Table 4: Training summary (optimized config, 2 epochs).

Config	Init Loss	Final Loss	Acc.	Step
LeNet/MNIST	2.35	~ 0.15	$\sim 95\%$	42.6 ms
SCNN/CIFAR	2.35	~ 1.50	$\sim 45\%$	58.6 ms
MLP/Synth	2.30	~ 0.80	$\sim 70\%$	14.8 ms

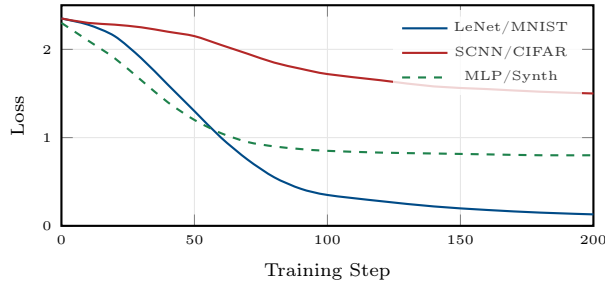


Figure 4: Training loss curves proving functional correctness.

5.2. Performance Characterization: The Latency Cliff

Figure 5 presents our central finding: optimizations that theoretically improve throughput (pipelining, async transfers) yield negligible or even *negative* returns on small workloads.

- **SimpleCNN (Moderate):** Shows a clear $\sim 3\%$ speedup (58.6ms $\rightarrow$ 57.0ms), where compute intensity is just enough to amortize scheduler costs.
- **MLP (Light):** Shows a regression (14.8ms $\rightarrow$ 15.1ms). The added overhead is quantifiable: the copy share of total step time increased from 2.55% to 3.77% in the optimized run. For these tiny steps, the cost of stream management outstrips any overlap benefit.

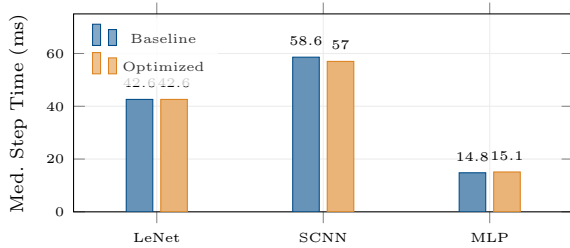


Figure 5: Step time comparison. Optimizations hurt the lightest workload (MLP).

5.3. Bottleneck Analysis: Why Optimizations Fail

To understand this efficiency gap, we dissected the execution trace, identifying three structural pathologies that define the latency-bound regime.

1. The Micro-Transfer Storm. We observed a staggering number of `cudaMemcpy(D2H)` calls—specifically, 5,600 per run for the MNIST baseline—transferring an average of just 18 bytes each. These correspond to individual scalar extractions (loss, accuracy, norm values). With a PCIe/Driver round-trip latency of $\sim 10\mu\text{s}$, a transfer of 18 bytes is entirely overhead-dominated. Crucially, these copies act as synchronization barriers, preventing the CPU from enabling deep queuing of kernels:

2. Occupancy & Utilization Paradox. As shown in Figure 6, average GPU utilization hovers at a dismal 14–15% for LeNet/MLP, despite the GPU being "active" for 99% of the step time. This is not a measurement error; it is the physical manifestation of *low-occupancy kernels*. Kernels like `conv2d_gradb` (bias gradient), `linear_bwd_gradb`, and `argmax_acc` launch with minimal grids (often a single block of 256 threads) to reduce just 16–128 values. This leaves 99% of the chip idle waiting for that single block to finish. We are paying the full fixed cost of kernel launch ($\sim 5\text{--}10\mu\text{s}$) for a computation that takes nanoseconds.

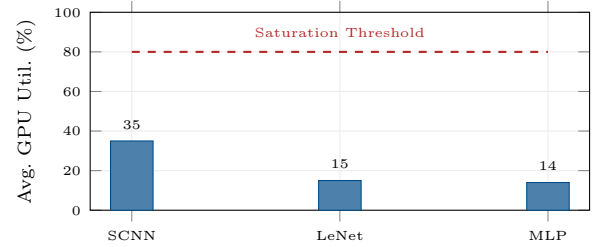


Figure 6: GPU Utilization. The 15% usage reflects idle hardware during low-occupancy kernel execution.

3. Launch Overhead Accumulation. For the MLP workload (14.8ms step), the cumulative time spent in kernel launch APIs and stream synchronization events becomes a significant fraction of the total frame budget. This explains why adding *more* orchestration (streams, events) for optimization led to a regression: we added overhead to a system that was already overhead-bound.

6. Key Kernel Implementations

In this section, we present the most performance-critical kernel implementations to illustrate the GPU programming decisions underlying the bottlenecks identified in Section 5.

6.1. Convolution Forward

The convolution forward kernel uses a *direct* (non-tiled) implementation where each thread computes one output pixel by iterating over the full filter window. This design is simple but does not exploit shared memory for data reuse across threads within a block.

```
--global__ void conv2d_fwd_kernel(
    const float* x, const float* w,
    const float* b, float* y,
    int n, int in_c, int in_h, int in_w,
    int out_c, int out_h, int out_w,
    int k_h, int k_w, int stride, int pad)
{
    const int idx = blockIdx.x * blockDim.x
        + threadIdx.x;
    const int total = n*out_c*out_h*out_w;
    if (idx >= total) return;
```

```

13  const int ow = idx % out_w;
14  const int oh = (idx / out_w) % out_h;
15  const int oc = (idx / (out_w*out_h))%out_c;
16  const int nn = idx / (out_w*out_h*out_c);
17
18  float acc = b ? b[oc] : 0.0f;
19  for (int ic = 0; ic < in_c; ++ic)
20      for (int kh = 0; kh < k_h; ++kh) {
21          const int ih = oh*stride + kh - pad;
22          if (ih<0 || ih>=in_h) continue;
23          for (int kw = 0; kw < k_w; ++kw) {
24              const int iw = ow*stride + kw - pad;
25              if (iw<0 || iw>=in_w) continue;
26              acc += x[((nn*in_c+ic)*in_h+ih)*in_w
27                     +iw]
28                    * w[((oc*in_c+ic)*k_h+kh)*k_w
29                       +kw];
30          }
31      }
32  y[idx] = acc;
33 }

```

Listing 1: Direct convolution forward kernel.

Design trade-off. Each thread performs $C_{in} \times K_H \times K_W$ multiply-accumulate operations. For LeNet’s first layer ($C_{in} = 1$, $K = 5$), this is only 25 FMAs per thread—moderate arithmetic intensity. For SimpleCNN’s deeper layers ($C_{in} = 32$, $K = 3$), it is 288 FMAs per thread—better but still below what tile-based implementations achieve via shared memory reuse.

6.2. Fused Softmax + Cross-Entropy

The softmax and cross-entropy computation are fused into a single kernel to avoid materializing the full probability distribution in global memory:

```

1  __global__ void softmax_ce_bwd_kernel(
2      const float* logits, const int* labels,
3      float* grad, float* loss,
4      int n, int classes)
5  {
6      const int sample = blockIdx.x;
7      if (sample >= n) return;
8
9      __shared__ float maxv, denom;
10     if (threadIdx.x == 0) {
11         float m = -1e20f;
12         for (int c=0; c<classes; ++c)
13             m = fmaxf(m, logits[sample*classes+c]);
14         maxv = m;
15         float s = 0.0f;
16         for (int c=0; c<classes; ++c)
17             s += expf(logits[sample*classes+c]-maxv);
18         denom = s;
19         if (loss) {
20             int y = labels[sample];
21             float p = expf(logits[sample*classes+y]
22                           - maxv) / denom;
23             loss[sample] = -logf(fmaxf(p, 1e-12f));
24         }
25     }
26     __syncthreads();
27     // Gradient: (softmax - one_hot) / n
28     int y = labels[sample];
29     for (int c = threadIdx.x; c < classes;
30          c += blockDim.x) {
31         float p = expf(logits[sample*classes+c]
32                       - maxv) / denom;
33         grad[sample*classes+c] =
34             (p - (c==y ? 1.0f : 0.0f)) / (float)n;
35     }
36 }

```

Listing 2: Fused softmax cross-entropy backward kernel.

Design note. This kernel uses one block per sample (n blocks), with intra-block parallelism across the class dimension. The max-finding and denominator computation are performed serialially by thread 0 and broadcast via shared memory—a design choice that serializes work but avoids a block-level reduction for the relatively small class dimension ($C = 10$).

7. Discussion

7.1. The Latency-Bound vs. Compute-Bound Dichotomy

The central finding of this work is that our CNN training pipeline is *not* compute-bound, memory-bound, or bandwidth-bound. It is *kernel-launch* and *synchronization* bound:

$$t_{\text{step}} \approx \underbrace{t_{\text{launch}} \cdot N_{\text{kernels}}}_{\text{launch overhead}} + \underbrace{t_{\text{sync}}}_{\text{D2H + events}} + \underbrace{t_{\text{compute}}}_{\text{actual GPU math}} \quad (1)$$

For our workloads, $t_{\text{launch}} \cdot N_{\text{kernels}} + t_{\text{sync}} \gg t_{\text{compute}}$ as evidenced by the 14–35% GPU utilization.

This is the worst kind of inefficiency for small-to-moderate models because it does not scale down gracefully: the overhead is a *fixed cost* per step that cannot be reduced by simply making the model smaller—in fact, smaller models make it *worse* because t_{compute} shrinks while the overhead remains constant.

7.2. Structural Origins in DL Training Pipelines

Modern deep learning training steps are composed of many small, heterogeneous operations:

1. Forward kernels (convolution, linear, activation, pooling)
2. Backward kernels (separate gradient kernels for data, weights, bias)
3. Reduction kernels (loss, norms, accuracy)
4. Optimizer kernels (SGD update per parameter group)
5. Metric extraction (scalar D2H copies)
6. Logging and serialization

When each of these is implemented as a separate small kernel—as is natural in a from-scratch implementation—the GPU receives many small pieces of work instead of fewer large ones. **GPUs are designed for the latter.**

7.3. Remediation Strategies

Based on our analysis, we identify four categories of remediation, mapped directly to the bottlenecks discovered:

A) Reduce kernel launch count (kernel fusion).

- Fuse bias gradient computation with weight gradient kernels as an “epilogue”—compute + reduce + write in one launch.
- Fuse multiple reduction operations (loss, norms, statistics) into a single multi-output kernel.
- Use CUDA Graphs to batch sequences of small kernels into a single launch command, amortizing driver overhead.

B) Eliminate tiny D2H transfers.

- Batch metrics: accumulate loss, accuracy, and norms on GPU for N steps, then copy one packed tensor instead of many individual scalars.
- Compute metrics less frequently (e.g., every 50–200 steps) to reduce D2H frequency.
- Use device-side final reductions to produce a single scalar on GPU, avoiding the partial-sum+CPU-aggregation pattern.

C) Increase per-launch parallelism.

- Increase batch size to enlarge kernel grid dimensions (more blocks $\rightarrow$ higher occupancy).
- Use gradient accumulation to simulate larger effective batch sizes.
- Consider tiled convolution implementations that use shared memory for data reuse, increasing arithmetic intensity per thread.

D) Accept overhead floor for small models.

For MNIST/LeNet and toy MLP workloads:

- Optimizations that add infrastructure complexity (async streams, double-buffering) are counterproductive.
- The best improvements come from *reducing* CPU/GPU chatter and *fusing* operations, not from async tricks.
- At 14.8 ms per step, even a 3 ms fixed overhead represents $\sim 20\%$ of wall-clock time, setting a hard floor on achievable speedup.

7.4. Comparison with Framework Implementations

Production deep learning frameworks (PyTorch, TensorFlow) address many of these bottlenecks automatically:

- **cuDNN** provides highly optimized, tiled convolution implementations with automatic algorithm selection (Winograd, FFT-based, or direct convolution depending on filter size and data dimensions).
- **Kernel fusion** in frameworks like PyTorch 2.0 (`torch.compile`) automatically fuses sequences of element-wise operations and small reductions.
- **CUDA Graphs** are used by frameworks to capture and replay kernel sequences, reducing per-kernel launch overhead.

- **Lazy metric extraction:** frameworks typically defer `.item()` calls and use asynchronous CUDA streams to minimize synchronization.

Our from-scratch implementation intentionally forgoes these optimizations to expose the raw performance behavior and demonstrate *why* they exist.

7.5. Workload Size as the Master Variable

Figure 7 illustrates how fixed overhead affects different workload sizes disproportionately.

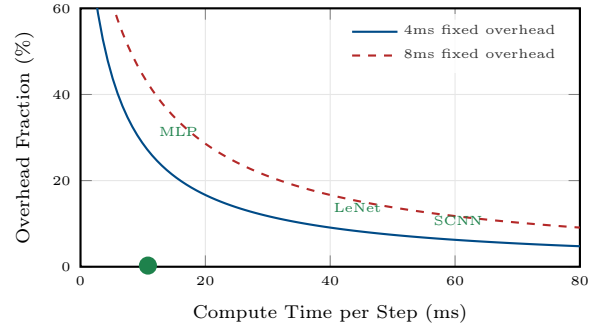


Figure 7: Overhead fraction vs. compute time for different fixed overhead levels. Lighter workloads suffer disproportionately.

8. Conclusion and Future Work

8.1. Summary

We presented CudaNN, a complete from-scratch CUDA C++ training platform for convolutional and fully connected neural networks. Through systematic profiling of six architecture–dataset configurations on an NVIDIA RTX 3050 Laptop GPU, we identified and analyzed eight concrete performance bottleneck categories.

Our central finding is that for small-to-moderate workloads, GPU training is *kernel-launch and synchronization bound*—not compute-bound, memory-bound, or bandwidth-bound. This manifests as a paradox: 96–99% of step time is spent in GPU execution paths, yet GPU utilization remains only 14–35%. The root causes are: (1) too many small kernel launches, (2) thousands of tiny D2H scalar transfers, (3) bias-gradient kernels with $\sim 1\%$ occupancy, and (4) frequent reduction kernels for per-step metrics.

Our optimization stack—comprising H2D streaming with double-buffered pinned memory, log-synchronization reduction, asynchronous checkpointing, and norm-logging throttling—provides modest improvement ($\sim 3\%$) for heavy workloads (SimpleCNN/CIFAR-10) but is neutral or slightly harmful for lightweight workloads (MLP/Synthetic). This demonstrates the *amortization failure* principle: optimizations that add

infrastructure complexity help only when there is sufficient compute to mask the extra overhead.

8.2. Future Work

Several directions could improve performance:

1. **Kernel fusion:** Fusing bias-gradient kernels with weight-gradient kernels and merging multiple reduction operations into single multi-output kernels.
2. **CUDA Graphs:** Capturing the full forward-backward-update sequence as a CUDA Graph to amortize launch overhead across the entire step.
3. **Tiled convolution:** Implementing shared-memory tiled convolution kernels (im2col + GEMM or direct tiled approaches) to increase arithmetic intensity and data reuse.
4. **Batched metric extraction:** Accumulating all per-step metrics on GPU and performing a single packed D2H transfer periodically.
5. **Larger-scale experiments:** Testing on models with higher computational intensity (e.g., ResNet on ImageNet) where the GPU is expected to transition from latency-bound to compute-bound, validating the overhead model of Equation ??.
6. **Mixed precision:** Using FP16/TF32 tensor cores on Ampere GPUs to increase effective throughput without changing the algorithmic structure.

- [7] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

References

- [1] A. Karpathy, “llm.c: LLM training in simple, raw C/CUDA,” GitHub repository, 2024. [Online]. Available: <https://github.com/karpathy/llm.c>
- [2] NVIDIA Corporation, “Nsight Compute and Nsight Systems Documentation,” 2023. [Online]. Available: <https://developer.nvidia.com/nsight-systems>
- [3] Y. Shen, T. Chen, et al., “Performance Characterization of DNN Training on Modern GPUs,” in *Proc. IEEE International Symposium on Workload Characterization (IISWC)*, 2022.
- [4] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [5] NVIDIA Corporation, “CUDA Programming Guide v12.x,” 2023. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- [6] NVIDIA Corporation, “cuDNN Developer Guide,” 2023. [Online]. Available: <https://docs.nvidia.com/deeplearning/cudnn/developer-guide/>